

RL Arm Motion

Code Reference Guide

CP493 Directed Research · Wilfrid Laurier University
Ranjot Sandhu · 2026

This document describes every source file in the project — what it does, what classes it contains, and how it fits into the overall system. Use it as a map when reading the code.

System Architecture at a Glance

Layer	Files	Role
Configuration	config/arm_config.py	All arm physical parameters in one place
Environment	environments/task_env.py	Physics, reward, observation, success logic — the core
Training	training/ppo_trainer_wrapper.py training/curriculum_callback.py	SAC training loop + adaptive curriculum
Models	models/trainers.py models/agents/sac_agent.py	SB3 model creation and hyperparameters
GUIs	gui/__main__.py gui/training_gui.py gui/app.py	Launcher, Training Dashboard, Interactive Arm
Utilities	utils/muscle_model.py utils/arm_kinematics.py	Hill muscle physics, forward kinematics
Validation	validation/fitts_law.py validation/power_law.py	Fischer 2021 scientific benchmarks
Scripts	scripts/train_fischer_session.py run_auto_search.py	Headless training CLI + automated seed search

1. Configuration

ArmConfiguration

src/rl_armMotion/two_d/config/arm_config.py

Lines of code: 246

Purpose

Stores every physical parameter of the arm in one place so nothing is hardcoded. Think of it as the 'blueprint' file — link lengths, masses, joint limits, damping, initial angles.

What it does

- Defines the **ArmConfiguration** dataclass with all arm physical properties.
- Provides **save/load** methods so configurations can be written to JSON and re-loaded.
- Sets the defaults: 2 DOF, link lengths [1.0, 0.8] m, shoulder at [-90°, 0°] home pose.
- Used by every other module — the environment, GUIs, and kinematics all read from this.

Key Classes & Functions

Class / Function	What it does
ArmConfiguration	Dataclass holding all arm parameters (link lengths, masses, joint limits, damping, initial angles). Default v

Note: Changing values here affects the entire system. Never change during a training run.

2. Environments

ArmTaskEnv (core environment)

src/rl_armMotion/two_d/environments/task_env.py

Lines of code: 926

Purpose

This is the most important file in the project. It is the Gymnasium-compatible environment that the SAC agent interacts with during training. It defines the physics, the reward, the observation, and the success condition.

What it does

- **Physics step:** updates joint angles and velocities each timestep — either via velocity commands (2D) or Hill-type muscle forces (4D).
- **Observation (11D):** [$\sin \theta_1$, $\cos \theta_1$, $\sin \theta_2$, $\cos \theta_2$, ω_1 , ω_2 , height_error, orientation_error, gradient_norm, in_goal_region, hold_progress].
- **10-term shaped reward:** approach reward, orientation reward, velocity penalty, smoothness penalty, gradient bonus, in-zone bonus, hold-growth bonus, waypoint advance bonus, waypoint touch bonus, terminal success bonus.
- **Waypoint navigation:** agent visits waypoints in order — intermediate ones advance on touch, the final one requires a 20-step hold.
- **Hold criterion:** position within tolerance AND orientation within 12° AND velocity < 0.3 rad/s for hold_steps_required consecutive steps.
- **Adaptive curriculum hooks:** set_goal_tolerance() and set_hold_steps_required() let the curriculum callbacks adjust difficulty during training.
- **ActionSmoother wrapper:** applies EMA smoothing to actions at eval/viz time (NOT during training).

Inputs / Outputs

Action in	4D float [0,1] muscle activations OR 2D float [-1,1] joint velocities
Observation out	11D float32 numpy array
Reward out	Scalar float (shaped, up to ~50,000 for a perfect episode)
Info dict	goal_reached, current_waypoint_index, hold_counter, hold_steps_required, joint_angles, end_effector_position, ...

Key Classes & Functions

Class / Function	What it does
ArmTaskEnv	Main Gymnasium environment. All training, evaluation, and GUI simulation runs through this class.
ActionSmoother	gym.ActionWrapper that applies EMA smoothing to actions — used only in the interactive GUI simulation.

Note: Never change the observation space shape or reward weights without re-training from scratch.

SimpleArmEnv (template / reference)

src/rl_armMotion/two_d/environments/simple_arm.py

Lines of code: 119

Purpose

A minimal 7-DOF arm template environment that shows the standard Gymnasium structure. Not used in training — kept as a reference scaffold for future 3D work.

What it does

- Shows the standard reset/step/render pattern for a Gymnasium environment.
- Useful as a starting point for extending the project to 3D or more DOF.

Key Classes & Functions

Class / Function	What it does
SimpleArmEnv	Template 7-DOF arm environment. Not used in the current 2-DOF training pipeline.

Note: This file is not used in any training or GUI workflow. It is a scaffold for future work.

3. Graphical User Interfaces

Unified Launcher (entry point)

src/rl_armMotion/two_d/gui/__main__.py

Lines of code: 293

Purpose

The front door of the project. Running 'python -m rl_armMotion.two_d.gui' opens a small launcher window that lets you start the Training GUI and the Interactive Arm GUI without using the command line.

What it does

- Spawns the Training GUI and Interactive Arm GUI as **independent subprocesses** so each gets its own Tkinter window.
- Form fields for algorithm (SAC/PPO/A2C), timestep budget, and save directory are pre-filled with Fischer 2021 defaults.
- Closing the launcher does NOT kill running training sessions — they keep going in the background.
- Tracks live PIDs in a status bar so you can see what's running.

Inputs / Outputs

Launch command	<code>python -m rl_armMotion.two_d.gui</code>
-----------------------	---

Key Classes & Functions

Class / Function	What it does
LauncherApp	Tkinter window with buttons to spawn both GUIs. Manages subprocess PIDs.
main()	Entry point — called by 'python -m rl_armMotion.two_d.gui'.

Training Dashboard GUI

```
src/rl_armMotion/two_d/gui/training_gui.py
```

Lines of code: 1154

Purpose

The training control panel. Lets you start a SAC training run and watch it live — reward curves, loss curves, arm pose, and curriculum progress all update in real time.

What it does

- **Live reward / loss / entropy plots** update every 100 training steps.
- **Live arm pose panel** shows where the end-effector is during training.
- **Curriculum metrics panel:** current goal tolerance, curriculum stage, rolling success rate.
- **Goal mode selector:** Direction preset (EAST/WEST/NORTH), Single-Point click, or Waypoint sequence click.
- **Save Model & Results button:** saves model .zip, training_history.csv, training_stats.json, Fitts PNG, Power Law PNG.
- **Run Validators button:** runs Fitts' Law and 2/3 Power Law sweeps on the loaded model.
- Uses a background thread for training so the GUI stays responsive.

Inputs / Outputs

Start training	Click 'Start Training' — spawns a background thread calling RLTrainerWithMetrics.train()
Save output	project_assets/outputs/<chosen_dir>/sac_model.zip + CSV + JSON + PNGs

Key Classes & Functions

Class / Function	What it does
TrainingGUI	Main Tkinter window. Manages the training thread, live plots, and all UI controls.

Interactive Arm GUI

```
src/rl_armMotion/two_d/gui/app.py
```

Lines of code: 1210

Purpose

The simulation and visualisation window. Load a trained model and watch the arm navigate to waypoints. Also supports manual joint control, parameter tuning, and motion recording.

What it does

- **Load Model button:** loads a .zip model file and sets the actuation mode automatically.
- **Run Simulation button:** runs the trained policy in the environment and animates the arm.
- **Manual control:** sliders for direct joint angle control.
- **Motion recording/playback:** record a manual sequence and play it back.

- **Torque plots:** shoulder and elbow torque vs time chart.
- **Live metrics panel:** hold counter, waypoint index, end-effector position.

Inputs / Outputs

Load	Select .zip model file via file dialog
Simulate	Runs <code>env.step()</code> in a loop, renders arm pose at ~30 fps

Key Classes & Functions

Class / Function	What it does
ArmControllerGUI	Main Tkinter application. Handles model loading, simulation loop, rendering, and manual controls.

Note: Uses ActionSmoother EMA wrapper during simulation for smoother visual motion.

4. Models & Agents

RLTrainer (core trainer)

`src/rl_armMotion/two_d/models/trainers.py`

Lines of code: 303

Purpose

The thin wrapper around Stable-Baselines3 that creates the SAC (or PPO/A2C) model, runs `.learn()`, saves, and loads. Every training run goes through this.

What it does

- Creates a Stable-Baselines3 model (SAC by default) with the correct hyperparameters.
- Provides **`train()`**, **`evaluate()`**, **`save()`**, and **`load()`** methods.
- Supports a linear learning-rate schedule (pickle-safe for SB3 compatibility).
- All algorithm choices (SAC/PPO/A2C) are routed through this single interface.

Inputs / Outputs

Input	ArmTaskEnv instance + algorithm name + hyperparams dict
Output	Trained SB3 model saved as .zip

Key Classes & Functions

Class / Function	What it does
RLTrainer	Creates and manages the SB3 model. Used by RLTrainerWithMetrics and <code>train_fischer_session.py</code> .
BaseTrainer	Abstract base class defining the trainer interface.
LinearDecaySchedule	Pickle-safe linear LR schedule dataclass.

SACAgent

`src/rl_armMotion/two_d/models/agents/sac_agent.py`

Lines of code: 75

Purpose

Holds the SAC hyperparameters that match Fischer et al. (2021). This is the only agent configuration that successfully solves the 3-waypoint task.

What it does

- Stores the Fischer 2021 SAC hyperparameters: lr=3e-4, batch=256, gamma=0.99, buffer=1M, ent_coef=auto, tau=0.005.
- Documents why each hyperparameter was chosen (with reference to the paper).
- Motor babbling note: SB3's SAC starts with random exploration automatically — no separate babbling phase needed.

Key Classes & Functions

Class / Function	What it does
SACAgent	Hyperparameter store for SAC. get_hyperparams() returns the Fischer 2021 config dict.

Note: PPO and A2C were tested and do not reliably solve the hold criterion. SAC is the only working algorithm.

PPOAgent

src/rl_armMotion/two_d/models/agents/ppo_agent.py

Lines of code: 48

Purpose

PPO hyperparameter wrapper. Available in the GUI but does not achieve reliable 3/3 waypoint completion.

What it does

- Standard PPO config: lr=3e-4, n_steps=2048, batch=64, epochs=10, gamma=0.99.
- Available for experimentation but SAC should always be preferred for this task.

Key Classes & Functions

Class / Function	What it does
PPOAgent	PPO hyperparameter store.

A3CAgent

src/rl_armMotion/two_d/models/agents/a3c_agent.py

Lines of code: 51

Purpose

A2C/A3C hyperparameter wrapper. Available in the GUI for experimentation.

What it does

- Uses A2C from Stable-Baselines3 (true async A3C is not supported by SB3).
- Standard config: lr=7e-4, n_steps=5, gamma=0.99.

Key Classes & Functions

Class / Function	What it does
A3CAgent	A2C/A3C hyperparameter store.

GUICallback (SB3 training callbacks)

src/rl_armMotion/two_d/models/callbacks.py

Lines of code: 227

Purpose

Stable-Baselines3 callbacks that extract metrics from the training loop and pass them to the GUI. Without these, the Training GUI would have no live data.

What it does

- **GUICallback**: fires every N steps, extracts episode rewards, losses, and entropy, pushes them into a queue for the GUI thread to read.
- **MetricsTracker**: accumulates a rolling history of training metrics.
- **PPOMetricsCallback**: PPO-specific metric extraction (policy loss, value loss, explained variance).

Key Classes & Functions

Class / Function	What it does
GUICallback	Base SB3 callback — collects episode data and pushes to a thread-safe queue every check_freq steps.
MetricsTracker	Rolling window accumulator for reward, loss, distance statistics.
PPOMetricsCallback	PPO-specific extension of GUICallback.

5. Training Infrastructure

RLTrainerWithMetrics (main training pipeline)

```
src/rl_armMotion/two_d/training/ppo_trainer_wrapper.py
```

Lines of code: 556

Purpose

The bridge between the raw SB3 trainer and the rest of the project. This is what train_fischer_session.py and the Training GUI both use. It automatically attaches all curriculum callbacks and streams live metrics.

What it does

- **Auto-attaches AdaptiveCurriculumCallback** whenever SAC is used — no manual setup needed.
- **Auto-attaches HoldCurriculumCallback** — starts hold requirement at 10 steps, graduates to 15 then 20.
- **Streams metrics** (reward, loss, entropy, hold progress, curriculum stage) through a thread-safe queue to the GUI.
- **save_model_and_results()** saves model .zip, training_history.csv, and training_stats.json.
- The **TrainingGUICallback** (inner class) is what actually fires every step and extracts data.

Inputs / Outputs

Input	ArmTaskEnv + total_timesteps + algorithm + optional hyperparams
Output	Trained model in memory + optional save to disk

Key Classes & Functions

Class / Function	What it does
RLTrainerWithMetrics	Main training wrapper. Attach curriculum callbacks, stream metrics, save results.
TrainingGUICallback	SB3 callback that extracts per-step data and feeds the metrics pipeline.
PPOTrainerWithMetrics	Backward-compatible PPO-only alias.

Curriculum Callbacks

```
src/rl_armMotion/two_d/training/curriculum_callback.py
```

Lines of code: 505

Purpose

Three SB3 callbacks that make the training task gradually harder as the agent improves. This is the Fischer et al. 2021 curriculum methodology — without it the agent cannot learn to hold precisely.

What it does

- **AdaptiveCurriculumCallback:** starts goal tolerance at 0.60 m (very easy to enter), shrinks it by 20% each time the rolling 50-episode success rate exceeds 80%, down to 0.02 m (very precise). Matches Fischer 2021 exactly.
- **HoldCurriculumCallback (new):** starts hold_steps_required at 10 (easy hold), advances to 15 then 20 as the agent achieves >60% success. Solves the 2/3 → 3/3 bottleneck.
- **EvalBasedCurriculumCallback:** alternative curriculum that advances based on periodic deterministic evaluation rather than stochastic training success rate.
- All three push updates to the environment via set_goal_tolerance() / set_hold_steps_required() hooks.

Key Classes & Functions

Class / Function	What it does
AdaptiveCurriculumCallback	Shrinks position tolerance from 0.60 m → 0.02 m as success rate rises. Fischer 2021 protocol.
HoldCurriculumCallback	Increases hold requirement from 10 → 15 → 20 steps as agent learns. New in this project.
EvalBasedCurriculumCallback	Curriculum based on periodic deterministic evaluation rather than training rollouts.

Note: Both AdaptiveCurriculumCallback and HoldCurriculumCallback run simultaneously — they operate independently on different aspects of the task.

6. Utilities

Hill-Type Muscle Model

src/rl_armMotion/two_d/utis/muscle_model.py

Lines of code: 274

Purpose

Implements the biomechanically realistic muscle physics used in muscle actuation mode. Instead of directly commanding joint velocities, the policy commands muscle activation levels (0–1), and this module converts them to forces and then torques.

What it does

- **Force equation:** $F = \text{activation} \times F_{\text{max}} \times f_L(\text{length}) \times f_V(\text{velocity})$.
- **f_L (force-length curve):** peaks at optimal fibre length, drops off for shorter/longer fibres. Gaussian shape.
- **f_V (force-velocity curve):** Hill's hyperbolic equation — force decreases as muscle shortens faster (concentric), increases as it lengthens (eccentric).
- **Antagonist pairs:** each joint has an extensor and a flexor muscle — $\text{net torque} = (F_{\text{extensor}} - F_{\text{flexor}}) \times \text{moment_arm}$.
- This naturally produces smooth, biologically plausible motion — the policy doesn't need a smoothness reward term.

Inputs / Outputs

Input	activation [0,1], current fibre length, current fibre velocity
Output	Force in Newtons → converted to torque by ArmTaskEnv

Key Classes & Functions

Class / Function	What it does
MuscleParameters	Dataclass: F_max, optimal length, moment arm, max velocity, pennation angle per muscle.
HillTypeMuscle	Computes muscle force given activation, current length and velocity. <code>apply_to_joint()</code> returns net torque.

Note: This is the same muscle model used in Fischer et al. (2021) and in OpenSim / MuJoCo musculoskeletal simulations.

Arm Kinematics

`src/rl_armMotion/two_d/utils/arm_kinematics.py`

Lines of code: 306

Purpose

Forward kinematics, joint control, and motion recording utilities. Computes where the end-effector is given joint angles.

What it does

- **ArmKinematics.forward_kinematics()**: given joint angles, returns (x, y) positions of each joint using standard 2D rotation matrices.
- **ArmController**: wraps kinematics with joint limit enforcement and velocity integration.
- **MotionRecorder**: records a sequence of arm states to a list; can save/load as JSON for playback in the Interactive GUI.

Key Classes & Functions

Class / Function	What it does
ArmState	Dataclass: joint angles, velocities, link endpoint positions, timestamp.
ArmKinematics	<code>forward_kinematics()</code> → 2D joint positions. <code>inverse_kinematics()</code> stub for future use.
ArmController	Integrates velocity commands, enforces joint limits, delegates to ArmKinematics.
MotionRecorder	Records/replays sequences of ArmState for the GUI playback feature.

Visualization Utilities

`src/rl_armMotion/two_d/utils/visualization.py`

Lines of code: 402

Purpose

Matplotlib and Plotly helpers for rendering the arm, plotting trajectories, and creating training dashboards.

What it does

- **ArmVisualizer.render_arm()**: draws the 2-link arm on a Matplotlib axes given joint angles.
- **ArmVisualizer.plot_trajectory()**: plots end-effector path over an episode.
- **SimulationVisualizer**: real-time animation loop used by the Interactive GUI.
- Also contains Plotly-based comparison dashboards (used in legacy notebooks).

Key Classes & Functions

Class / Function	What it does
ArmVisualizer	Matplotlib arm renderer and trajectory plotter.
SimulationVisualizer	Real-time animation loop for the GUI.

Note: The Plotly dependency (plotly) is optional — comment it out in requirements.txt if not needed.

Parallel Environment Utilities

src/rl_armMotion/two_d/utils/parallel_env.py

Lines of code: 191

Purpose

Utilities for running multiple environment instances in parallel using multiprocessing. Used for data collection and future distributed training experiments.

What it does

- `run_single_simulation()`: runs one episode of any Gymnasium environment in a subprocess.
- **ParallelEnvironmentRunner**: manages a pool of worker processes, each running an independent simulation.
- **VectorEnvironment**: wraps multiple environments for synchronous stepping (similar to SB3 DummyVecEnv but custom).

Key Classes & Functions

Class / Function	What it does
SimulationResult	Dataclass: env_id, episode_reward, episode_length, final observation, info dict.
ParallelEnvironmentRunner	Manages a multiprocessing Pool of simulation workers.
VectorEnvironment	Synchronous multi-env wrapper for step/reset.

Note: Not used in the main training pipeline — the main SAC training uses SB3's built-in single-env loop. This is for custom data collection experiments.

7. Validation Harnesses

Fitts' Law Validator

src/rl_armMotion/two_d/validation/fitts_law.py

Lines of code: 569

Purpose

Tests whether the trained arm policy reproduces the classical human speed-accuracy trade-off (Fitts' Law). This is one of the two scientific benchmarks from Fischer et al. (2021) that prove the policy behaves like a biological arm.

What it does

- **Fitts' Law**: Movement Time = $a + b \times ID$, where $ID = \log_2(2 \times \text{Distance} / \text{Width})$. Humans follow this law across all target sizes and distances.
- Sweeps 12 target distances $\times$ 3 widths = 36 conditions, runs 10 trials each by default.
- Measures the time from movement onset to first entering the goal zone.
- Fits OLS regression on MT vs ID, reports R^2 and the a, b coefficients.
- Saves results to **fitts_law.json** and a scatter/regression **fitts_law.png**.
- Fischer et al. achieved $R^2 = 0.9986$ on their 7-DOF arm — a result close to this is expected.

Inputs / Outputs

Input	Trained SB3 model + ArmTaskEnv instance
Output	fitts_law.json (all trial data) + fitts_law.png (regression plot)

Key Classes & Functions

Class / Function	What it does
FittsLawValidator	Main harness — sweeps conditions, runs rollouts, fits regression.
FittsLawResult	Stores all trial data and regression coefficients. save_json() and plot() methods.
FittsLawCondition	One (distance, width) test condition.
FittsLawTrial	One deterministic rollout result for one condition.

2/3 Power Law Validator

src/rl_armMotion/two_d/validation/power_law.py

Lines of code: 555

Purpose

Tests whether the trained policy reproduces the 2/3 Power Law — the relationship between hand speed and path curvature observed in all human arm movements. The second Fischer 2021 benchmark.

What it does

- **2/3 Power Law:** $V = K \times C^{(-1/3)}$, i.e. the arm naturally slows down on tight curves. Humans follow this without being explicitly trained to.
- Runs 20 circular arc trajectories through the workspace.
- Extracts per-step end-effector speed V and curvature C by central differences.
- Fits log-log regression: $\log V = \log K + \text{slope} \times \log C$. Slope should be close to $-1/3$.
- Saves results to **power_law.json** and a log-log scatter **power_law.png**.
- Fischer et al. reported $R = 0.84$ on their arm.

Inputs / Outputs

Input	Trained SB3 model + ArmTaskEnv instance
Output	power_law.json + power_law.png

Key Classes & Functions

Class / Function	What it does
PowerLawValidator	Main harness — runs arc trials, computes V and C , fits regression.
PowerLawResult	Stores all trial data and regression. save_json() and plot() methods.
PowerLawTrial	One trajectory trial result.

8. Scripts

train_fischer_session.py (headless training CLI)

scripts/train_fischer_session.py

Lines of code: 268

Purpose

Command-line script for running a full Fischer 2021 training session without opening any GUI. Used by run_auto_search.py to train each seed.

What it does

- Parses CLI flags: --timesteps, --actuation-mode, --goal-direction, --waypoints, --save-dir, --num-threads.

- Builds ArmTaskEnv, wraps it in RLTrainerWithMetrics (which auto-attaches both curriculum callbacks).
- Trains for the requested number of steps, then saves model + CSV + JSON.
- Runs Fitts' Law and 2/3 Power Law validators on the trained model and saves PNGs.
- All output logged to training_log.txt in the save directory.

Inputs / Outputs

CLI example	python scripts/train_fischer_session.py --timesteps 500000 --actuation-mode muscle --waypoints '2.3,-1.0;1.0,1.5;2.5,0
Output files	sac_model.zip, training_history.csv, training_stats.json, fitts_law.png, power_law.png, training_log.txt

Key Classes & Functions

Class / Function	What it does
parse_args()	Defines all CLI flags with defaults and help text.
main()	Orchestrates env construction → training → saving → validation.

run_auto_search.py (automated champion search)

run_auto_search.py

Lines of code: 303

Purpose

The automated batch search that runs 5 seeds in parallel and stops as soon as one achieves 3/3 waypoints. This is the main script running during the research phase.

What it does

- Trains 5 seeds in parallel, each calling train_fischer_session.py as a subprocess.
- **Early killing:** at 250K steps, any seed with reward < 0 is killed — saves ~45 min per bad seed.
- **Thread pinning:** each seed gets 3 PyTorch threads (total = 15 threads on 24 cores), preventing CPU fights.
- **Auto-evaluation:** after each batch, evaluates all completed models with deterministic rollouts.
- **Winner analysis:** when 3/3 found — runs 5 consistency checks, evaluates at 3 tolerances (0.6/0.4/0.2 m), copies model to CHAMPION/, writes REPLICATION_REPORT.md.
- **Resume-safe:** on restart, skips seeds that already have a saved model.

Inputs / Outputs

Run	python run_auto_search.py
Output	project_assets/outputs/Parallel_Seeds/seed_NNN/ per seed
Champion	project_assets/outputs/CHAMPION/sac_model.zip + REPLICATION_REPORT.md

Key Classes & Functions

Class / Function	What it does
get_latest_reward()	Parses a seed's stdout.log to extract the latest ep_rew_mean.
evaluate_model()	Loads a model and runs one deterministic rollout, returns waypoints reached.
analyze_winner()	Full consistency analysis + CHAMPION copy + REPLICATION_REPORT generation.
run_seed()	Launches one training subprocess, monitors it, kills if below threshold.
main()	Outer batch loop — runs batches until champion found or max_batches reached.

Note: Reduce SEEDS_PER_BATCH from 5 to 3 if the computer runs slowly during training.